Hàm và cấu trúc chương trình (Functions and Program Structure)

Nhập môn lập trình

Vũ Đình Khôi

Nội dung

- Hàm
  - Hàm là gì?
  - Xây dựng hàm
  - Sử dụng hàm
  - Truyền tham số
  - Hàm trả về giá trị và hàm không trả về giá trị
  - Khai báo hàm nguyên mẫu
- Cấu trúc chương trình
  - Cấu trúc chung một chương trình
  - Tầm vực biến
  - Khối lệnh

Khái niệm

- Hàm
  - Thực hiện một chức năng (function) nào đó,
  - Chức năng này bao gồm một đoạn code (subroutine) có thể được sử dụng nhiều lần [ở các vị trí khác nhau trong chương trình].
  - Subroutine được gọi là function (C/C++) hoặc method (Java)
- Các lợi ích của hàm
  - Chia để trị: các hàm có thể chia nhỏ các công việc tính toán lớn (large computing tasks), phức tạp thành các công việc nhỏ hơn và đơn giản hơn.
  - Tránh sử dụng các đoạn code trùng lặp
  - Tái sử dụng phần mềm: có thể sử dụng lại các hàm trong các chương trình khác [bằng cách đóng gói thành các thư viện]

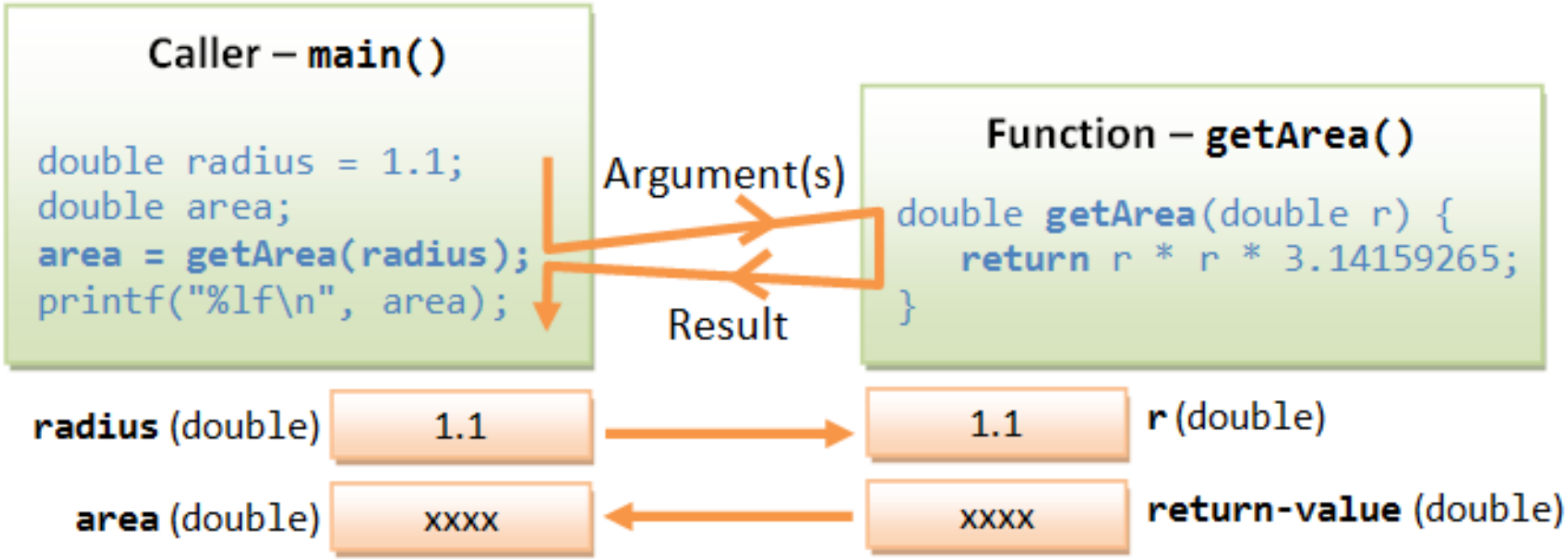
Xây dựng hàm

```
returnValueType functionName (parameterList) {
    functionBody;
    [return [value|expression]]
}
```

- functionName
  - Tên hàm [đặt giống tên biến, nhưng có ý nghĩa]. Thường là động từ hoặc cụm động từ
  - Ví dụ: getArea(), calculateArea(), setRadius(), moveDown(), hoặc isPrime()...
- returnValueType
  - kiểu dữ liệu của giá trị trả về [sau khi hàm xử lý hoặc tính toán]. Ví dụ: int, double
- parameterList
  - danh sách các tham số truyền vào hàm. Ví dụ: double getArea(double radius)

Sử dụng hàm

- Caller
  - Đoạn chương trình trong đó có sử dụng hoặc gọi (call) hàm
  - Có nhiệm vụ truyền giá trị các tham số (arguments) cho hàm
  - Hàm nhận các tham số, sau khi tính toán, sẽ trả kết quả cho caller



Ví dụ

```
1   /* Ham (function) va cau truc chuong trinh */
2   #include <stdio.h>
3
4   const float PI = 3.1416;
5
6   // Khai bao ham (Function declaration/ Function prototype)
7   double getArea(double radius);
8
9   // Dinh nghia cac ham (Function definition)
10  double getArea(double radius) {
11      double area = radius * radius * PI;
12      return area;
13  }
```

Tham số hình thức (parameter)

Ví dụ

```
15 int main() {
16     double r1 = 1.0, area1, area2;
17
18     // Goi ham, truyen tham so va nhan ket qua tra ve
19     area1 = getArea(r1);
20     printf("Area 1: %.2lf\n", area1);
21
22     // Goi ham, truyen tham so truc tiep so va nhan ket qua tra ve
23     area2 = getArea(2.0);
24     printf("Area 2: %.2lf\n", area2);
25
26     // Goi ham, truyen tham so so va nhan ket qua tra ve truc tiep
27     printf("Area 3: %.2lf\n", getArea(3.0));
28
29     return 0;
30 }
```

Tham số thực (argument)

Hàm được gọi/sử dụng nhiều lần

Ví dụ

- Tính giá trị biểu thức

$$z = 1 + \frac{1+2}{2!} + \frac{1+2+3}{3!} + \cdots + \frac{1+2+3+\cdots+n}{n!}$$

- Chia bài toán phức tạp thành các cấu trúc/bài toán nhỏ hơn
  - Làm sao tính tổng

$$s_1 = 1, s_2 = 1+2, s_3 = 1+2+3, \ldots, s_n = 1+2+3+\cdots+n$$

  - Tính giai thừa (factorial)

$$1! = 1, 2! = 1 \times 2, 3! = 1 \times 2 \times 3, \ldots, n! = 1 \times 2 \times 3 \times \cdots \times n$$

  - Tính tổng các biểu thức con

$$z = \frac{s_1}{1!} + \frac{s_2}{2!} + \frac{s_3}{3!} + \cdots + \frac{s_n}{n!}$$

Ví dụ

```c
/* Tinh tong gia tri bieu thuc */
#include <stdio.h>

// Function prototype
int sum(int n);
double factorial(int n);

//Function definition
int sum(int n) {
    int i, s = 0;
    for (i = 1; i <= n; i++) {
        s += i;
    }
    return s;
}

double factorial(int n) {
    double f = 1;
    int i;
    for (i = 1; i <= n; i++) {
        f *= i;
    }
    return f;
}
```

Ví dụ

```c
int main() {
    int i, n = 10;
    double z = 0;

    for(i = 1; i <= n; i++) {
        int s = sum(i);
        double f = factorial(i);
        printf("n = %d\ts = %d\tf = %.0lf\n", i, s, f);
        z = z + s / f;  // z += sum(i) / factorial(i);
    }
    printf("z = %.4lf", z);
    return 0;
}
```

```
n = 1    s = 1    f = 1
n = 2    s = 3    f = 2
n = 3    s = 6    f = 6
n = 4    s = 10   f = 24
n = 5    s = 15   f = 120
n = 6    s = 21   f = 720
n = 7    s = 28   f = 5040
n = 8    s = 36   f = 40320
n = 9    s = 45   f = 362880
n = 10   s = 55   f = 3628800
z = 4.0774
```

Ví dụ

```c
/* Max function */
#include <stdio.h>

// Function prototype
int maximum(int, int);

// Function definition
int maximum(int a, int b) {
    return (a > b)? a : b;
}

int main() {
    printf("Max: %d\n", maximum(5, 8));

    int a = 4, b = 7, c = 9;
    printf("Max: %d\n", maximum(maximum(a, b), 9));

    return 0;
}
```

"void" return type

- Hàm thực thi một/nhiều hành động nào đó (ví dụ chỉ có các lệnh printf) và không trả về giá trị nào cho caller.
  - Có thể dùng lệnh return cuối hàm hoặc không (optional).

```c
/* Max function */
#include <stdio.h>

// Function definition
void showMenu() {
    int choice;
    do {
        printf("--- MY PROGRAM ---\n");
        printf("1. New...\n");
        printf("2. Open...\n");
        printf("3. Save...\n");
        printf("0. Exit.\n");
        printf("Enter your choice (1-4): ");
        scanf("%d", &choice);
    } while (choice);
    return;
}

void main() {
    showMenu();
}
```

Boolean functions

- Các hàm trả về giá trị 0 (False) hoặc khác 0 (True).

```c
/* Kiem tra so chan le */
#include <stdio.h>
// Function prototype/declaration
int isEven(int);  // so chan?
int isOdd(int number);  // so le?

// Function definition
int isEven(int number) {
    if (number % 2 == 0)
        return 1;
    else
        return 0;
}

int isOdd(int number) {
    return (number % 2 != 0);
}
```

```c
void main() {
    printf("%d\n", isEven(5));
    printf("%d\n", isEven(6));
    printf("%d\n", isOdd(7));
    printf("%d\n", isOdd(8));
}
```

D:\C\function_example_04.exe

```
0
1
1
0
```

Boolean functions

- Có thể viết gọn như sau

```
// Function definition
int isEven(int number) {
    return (number %2 == 0);
}

int isOdd(int number) {
    return !isEven(number);
}
```

Boolean functions

- Sử dụng kiểu dữ liệu bool
  - bool không phải là kiểu dữ liệu có sẵn (built-in data type). Từ “C99 standard” mới đưa vào
  - Phải sử dụng kiểu bool qua thư viện stdbool.h

```c
/* Kiem tra so chan le */
#include <stdio.h>
#include <stdbool.h>
// Function definition
bool isEven(int number) {
    if (number %2 == 0)
        return true;
    else
        return false;
}

bool isOdd(int number) {
    return !isEven(number);
}

void main() {
    printf("%d\n", isEven(5));
    printf("%d\n", isEven(6));
    printf("%d\n", isOdd(7));
    printf("%d\n", isOdd(8));
}
```

Truyền tham số (passing arguments)

- Có 2 cách truyền tham số cho một hàm:
  - Truyền bằng giá trị/trị số: gọi là tham trị (pass-by-value)
  - Truyền bằng chính một biến (địa chỉ của biến): gọi là tham chiếu/tham biến (pass-by-reference)
- Có thể hiểu
  - Tham trị: tương tự việc copy/clone giá trị của một biến. Do đó, không làm thay đổi biến ban đầu.
  - Tham chiếu: việc thay đổi giá trị sẽ thay đổi biến ban đầu.

Tham trị và tham chiếu

```c
/* Tham tri va tham chieu (Pass-by-value & Pass-by-reference) */
#include <stdio.h>

void changeNumber_1(int number) {
    number += 10;
    printf("Number after changed: %d\n", number);
}

void changeNumber_2(int *number) {
    *number += 10;  // truy cap qua dia chi bien number
    printf("Number after changed: %d\n", *number);
}

void main() {
    int a = 5, b = 7;
    changeNumber_1(a);  // truyen tham tri
    printf("Original number: %d\n", a);

    changeNumber_2(&b);  // truyen dia chi/tham chieu
    printf("Original number: %d\n", b);
}
```

Tham trị và tham chiếu

```c
/* Tham tri va tham chieu
(Pass-by-value & Pass-by-reference) */
#include <stdio.h>

void swap(int, int);

void swap(int a, int b) {
    int temp;
    temp = a;
    a = b;
    b = temp;
}

void main() {
    int a = 5, b = 7;
    printf("Before: a = %d, b = %d\n", a, b);
    swap(a, b);
    printf("After: a = %d, b = %d\n", a, b);
}
```

```
D:\C\function_example_07.exe
Before: a = 5, b = 7
After: a = 5, b = 7
```

Tham trị và tham chiếu

```c
/* Tham tri va tham chieu
(Pass-by-value & Pass-by-reference) */
#include <stdio.h>

void swap(int *a, int *b);

void swap(int *a, int *b) {
    int temp;
    temp = *a;
    *a = *b;
    *b = temp;
}

void main() {
    int a = 5, b = 7;
    printf("Before: a = %d, b = %d\n", a, b);
    swap(&a, &b);
    printf("After: a = %d, b = %d\n", a, b);
}
```

```
D:\C\function_example_07a.exe
Before: a = 5, b = 7
After: a = 7, b = 5
```

Câu lệnh return và exit

- return
  - Khi gặp câu lệnh này trong hàm, máy tính sẽ chuyển quyền điều khiển ra ngoài, không thực hiện các câu lệnh còn lại trong hàm.
- exit()
  - Chấm dứt thực hiện chương trình, trả điều khiển về hệ điều hành.

Thiết kế và lập trình module/function

Không nên thực hiện việc nhập n và hiển thị các kết quả trong hàm sum()

```c
/* Khong nen */
#include <stdio.h>
void sum() {
    int i, n, s = 0;
    printf("Enter n (n > 0): ");
    scanf("%d", &n);
    for (i = 1; i <= n; i++) {
        s += i;
    }
    printf("sum = %d", s);
}

void main() {
    sum();
}
```

Hàm sum() chỉ thực hiện công việc tính tổng và trả kết quả về qua lệnh return

```c
#include <stdio.h>
int sum(int n) {
    int i, s = 0;
    for (i = 1; i <= n; i++) {
        s += i;
    }
    return s;
}

void main() {
    int n;
    printf("Enter n (n > 0): ");
    scanf("%d", &n);
    int s = sum(n);
    printf("sum = %d", s);
}
```

Cấu trúc chương trình

```c
/* Cau truc chuong trinh (Structure of program) */

/* Khai bao cac thu vien (li
#include <stdio.h>

/* Khai bao cac cac hang so
 va bien toan cuc (global va
 */
const double PI = 3.1416;
int n = 10, m = 20;

/* Khai bao cac ham */
double calculate(int n);

/* Ham main */
int main() {
    // Cac bien cuc bo (local variables)
    double a;
    int n = 5;
    printf("Local variable: n = %d\n", n);
    a = calculate(n);  // n la bien cuc bo
    printf("a = %.2lf\n", a);
    printf("Global variable: m = %d\n", m);

    return 0;
}

/* Dinh nghia cac ham */
double calculate(int n) {
    return n * PI;
}
```

```
D:\C\function_example_09.exe
Local variable: n = 5
a = 15.71
Global variable: m = 20
```

Phạm vi của biến (scope of variables)

- Có 2 loại biến
  - Biến toàn cục (global): phạm vi ảnh hưởng trong toàn bộ chương trình
  - Biến cục bộ (local): phạm vi ảnh hưởng trong nội bộ một hàm hoặc trong một khối lệnh. Nếu khai báo biến trong cặp ngoặc {...} thì biến chỉ có phạm vi trong cặp ngoặc đó.
- Tại một vị trí trong trường trình, nếu có 2 biến toàn cục và cục bộ trùng tên → biến cục bộ sẽ được sử dụng.

Khối lệnh (block)

- Khối lệnh là một dãy các câu lệnh được bao bởi cặp dấu {...}
- Ngôn ngữ C xem một khối lệnh cũng như một lệnh riêng lẻ.
  - Chỗ nào viết được một lệnh → thì chỗ đó cũng đặt được một khối lệnh.
- Các khối lệnh có thể lồng nhau (nested blocks)
  - Bên trong một khối lệnh có thể có một khối lệnh khác.
  - Sự lồng nhau như thế là không hạn chế.
- Thân hàm cũng là một khối lệnh.

Khối lệnh (block)

```
1   #include <stdio.h>
2
3   int main() {
4       int n = 5;
5
6       if(n > 0) {
7           printf("Statement 1.\n");
8           printf("Statement 2.\n");
9           printf("Statement 3.\n");
10      }
11      else
12          printf("Statement 4.\n");
13
14      return 0;
15  }
```

Một khối lệnh

Một lệnh đơn

Bài tập

- Xem file *NMLT_Lab 07.pdf*